Lab 4 - DLX Decode

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

This project required making the decode stage out of the 5 stages for the DLX processor. This stage has several components including: register bank, registers, and a sign extender. This project entails receiving the instruction and parsing it to grab register files and sign extending the immediate value to 32 bits. Then passing the register contents, the PC+1 value, the sign extended immediate value, and the instruction set. The decode stages expects some signals from the memory write back stage to determine what registers to change.

Procedure

Modular Systems:

- We started off by making each block that would match the blocks we had made in our block diagram. (See Figure 1)
- We then tied them together
- We then made each test bench to verify each smaller piece of the bigger puzzle. (See Figures 2-3)
- Created _pkg files for each component to ease the use of them in later labs if needed
    - We also added the constants for all the op codes to the decode_reg_pkg.vhd file which is a place to store all our op codes for use in the future. We realized after that we wouldn't need them until the execute stage.

Challenges and Solutions:

- The primary difficulty lay in simulating the top-level DLX_processor.vhd file within Questa. However, we realized by just modifying the testbench from the fetch lab that we could easily test it. Therefore we were able to easily test the new decode stage. We also hope to continue to modify the test bench with the later stages.

Results

The final implementation showed us that the decode stage can correctly grab data from the registers and write back to the registers. It was able to use the fetch stage correctly and grab the correct registers from the instructions fetched. The top module was modified such that it could output certain signals for testing which made testing easier.

Figures

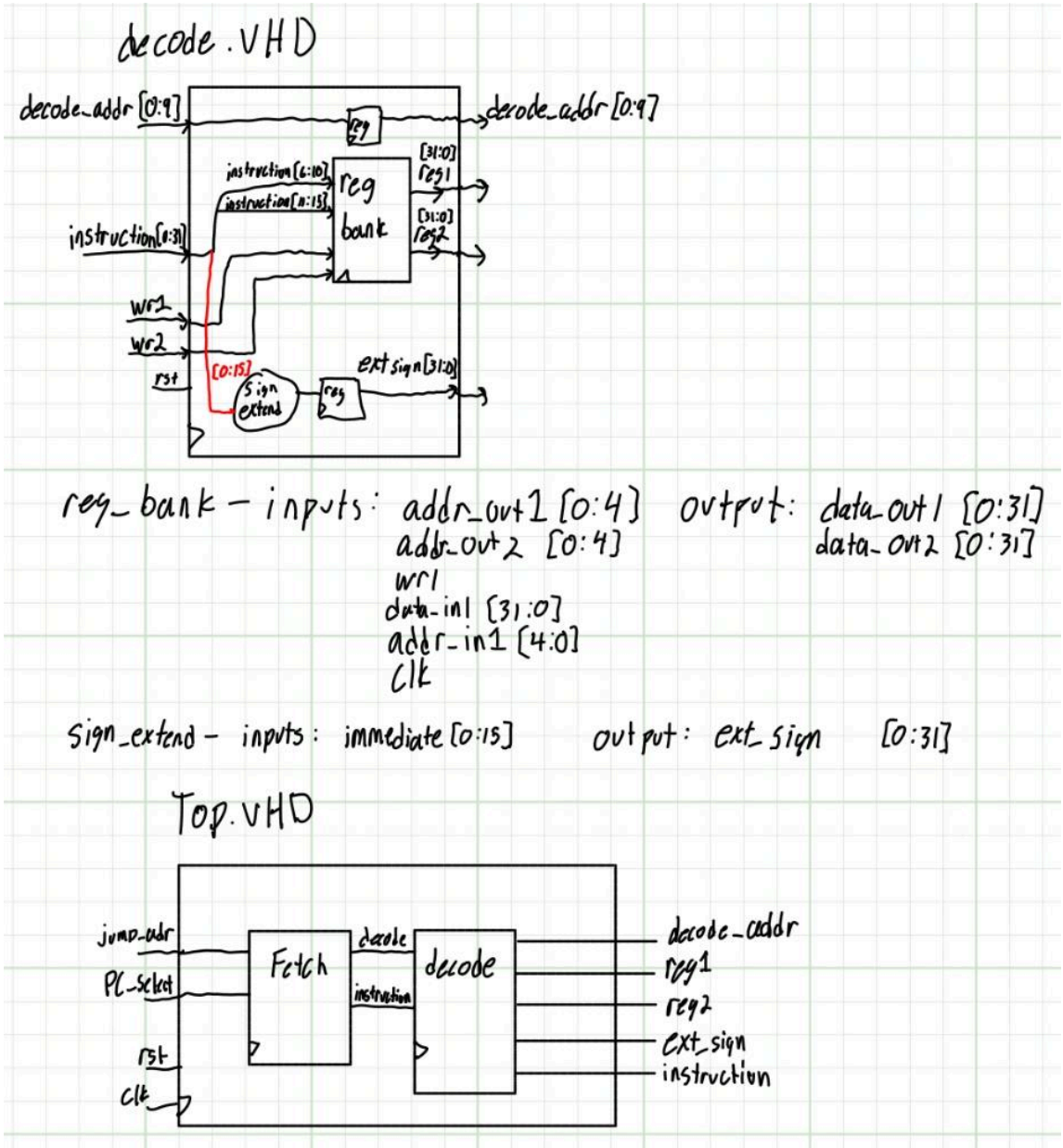

Figure 1: First Initial Block diagram for all the blocks we needed to implement and link together and top VHDL file.

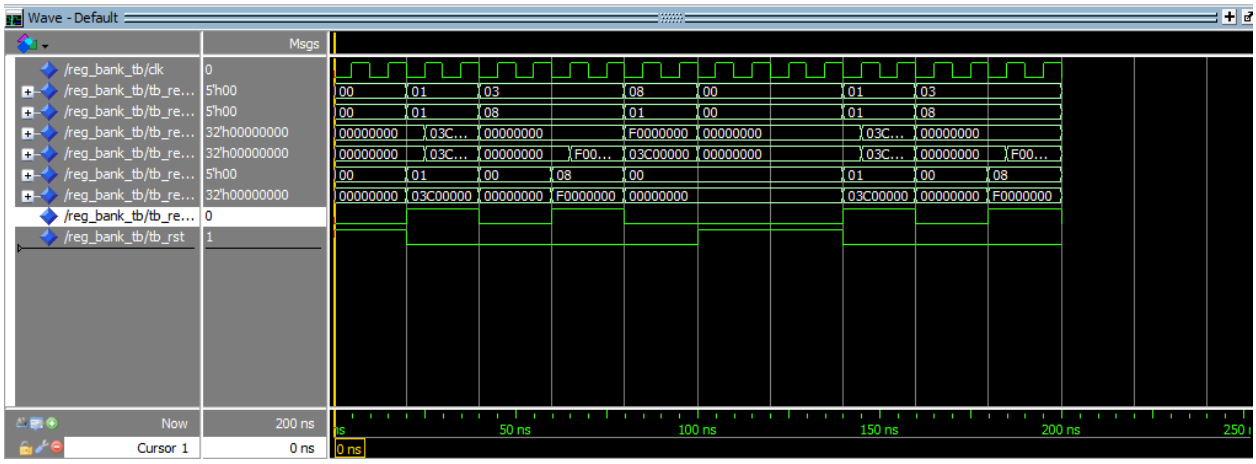

Figure 2: decode reg simulation

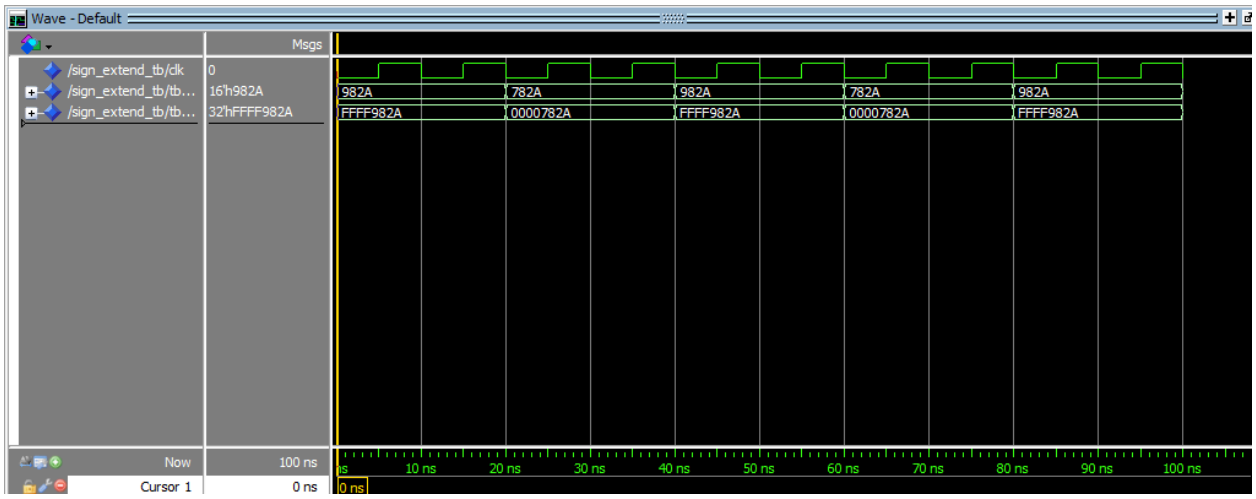

Figure 3: Register simulation

Figure 4: Sign extend simulation

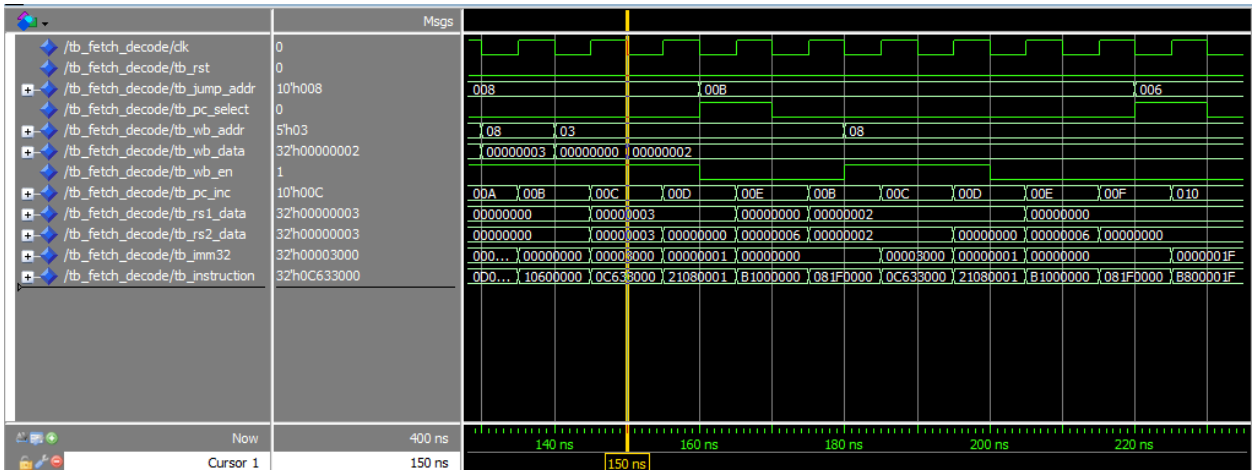

Figure 5: top module simulation

Conclusion

This lab provided valuable insights into the workflow on implementing different stages of a processor. This lab will help us in the future labs as we have a better understanding of how to pipeline the data and when and where to register the data. It has been a fun experience and we are learning a lot of hardware design flow through this.

Appendix

DLX_Processor.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;

library work;
use work.fetch_pkg.all;
--use work.decode_pkg.all;

entity DLX_Processor is
   generic (
       WIDTH       : integer := 10;
       INSTR_WIDTH : integer := 32
   );
   port (
       clk             : in  std_logic;
       rst             : in  std_logic;

       -- Branch/Jump Control (Driven by TB for now/Execute stage later)
       jump_addr_in    : in  std_logic_vector(WIDTH-1 downto 0);
       pc_mux_sel      : in  std_logic;

       -- Writeback Interface (Driven by TB for now/Writeback stage later)
       write_addr_in   : in  std_logic_vector(4 downto 0);
       write_data_in   : in  std_logic_vector(31 downto 0);
       write_en_in     : in  std_logic;

         pc_inc                : out   std_logic_vector(WIDTH-1 downto 0);
         rs1_data          : out   std_logic_vector(INSTR_WIDTH-1 downto
0);
         rs2_data          : out   std_logic_vector(INSTR_WIDTH-1 downto
0);
         imm32                 : out std_logic_vector(INSTR_WIDTH-1 downto
0);
         instruction       : out std_logic_vector(INSTR_WIDTH-1 downto 0)
   );
end entity DLX_Processor;
```

```
architecture structural of DLX_Processor is
   -- Decode outputs
   signal rd_addr     : std_logic_vector(4 downto 0);
   signal internal_pc_inc: std_logic_vector(9 downto 0);
    signal internal_instr :    std_logic_vector(31 downto 0);

begin

   -- Fetch Stage
   fetch_inst: fetch
       generic map (
           N => WIDTH,
           M => INSTR_WIDTH
       )
       port map (
           jump_addr   => jump_addr_in,
           pc_select   => pc_mux_sel,
           rst         => rst,
           clk         => clk,
           decode_addr => internal_pc_inc,
           instruction => internal_instr
       );

   -- Decode Stage
   decode_inst: entity work.decode
       generic map (
           FUNC_WIDTH => 6,
           ADDR_WIDTH => 5,
           DATA_WIDTH => 32
       )
       port map (
           clk             => clk,
           rst             => rst,
           instruction_in  => internal_instr,
           pc_inc          => internal_pc_inc,
           wb_data         => write_data_in,
           wb_addr         => write_addr_in,
           wb_en           => write_en_in,
           rs1_data        => rs1_data,
```

```
            rs2_data        => rs2_data,
            sign_ext_imm    => imm32,
            rd_addr_out     => rd_addr,
            pc_inc_out      => pc_inc,
                instruction_out => instruction
        );

end architecture structural;
```

decode.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.decode_reg_pkg.all;
use work.sign_extend_pkg.all;

entity decode is
    generic (
        FUNC_WIDTH : integer := 6;
        ADDR_WIDTH : integer := 5;
        DATA_WIDTH : integer := 32
    );
    port (
        clk             : in  std_logic;
        rst             : in  std_logic;

        -- From Fetch
        instruction_in  : in  std_logic_vector(31 downto 0);
        pc_inc          : in  std_logic_vector(9 downto 0);

        -- From Writeback
        wb_data         : in  std_logic_vector(DATA_WIDTH-1 downto 0);
        wb_addr         : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
        wb_en           : in  std_logic;
```

```
       -- To Execute
         instruction_out : out std_logic_vector(31 downto 0);
       rs1_data        : out std_logic_vector(DATA_WIDTH-1 downto 0);
       rs2_data        : out std_logic_vector(DATA_WIDTH-1 downto 0);
       sign_ext_imm    : out std_logic_vector(31 downto 0);
       rd_addr_out     : out std_logic_vector(ADDR_WIDTH-1 downto 0);
       pc_inc_out      : out std_logic_vector(9 downto 0)
   );
end entity decode;

architecture structural of decode is
   signal opcode : std_logic_vector(5 downto 0);
   signal rs1_addr, rs2_addr, rd_addr_r : std_logic_vector(4 downto 0);
   signal imm16 : std_logic_vector(15 downto 0);
    signal sign_ext :  std_logic_vector(31 downto 0);

   signal reg_dest_sel : std_logic_vector(1 downto 0); -- 00:
I-Type(20-16), 01: R-Type(15-11), 10: 31
   -- 00 is also used for instructions that don't write back (Stores,
Branches), since wb_en is 0 anyway.

begin
   opcode <= instruction_in(31 downto 26);
   rs1_addr <= instruction_in(25 downto 21);
   rs2_addr <= instruction_in(20 downto 16); -- Also acts as RD for I-Type
   rd_addr_r <= instruction_in(15 downto 11);
   imm16 <= instruction_in(15 downto 0);

    instr_reg  :   entity work.reggi
       generic map(
           N => 32
       )
       port map(
           data_in => instruction_in,
           rst   => rst,
           clk       => clk,
           data_out=> instruction_out
       );

   -- Sign Extender
```

```
sign_extER: sign_extend port map (
    input => imm16,
    output => sign_ext
);

 Sign_reg   :   entity work.reggi
    generic map(
        N => 16
    )
    port map(
        data_in => sign_ext,
        rst   => rst,
        clk   => clk,
        data_out=> sign_ext_imm
    );


-- Register File
reg_file: decode_reg
    generic map (DATA_WIDTH => 32, ADDR_WIDTH => 5)
    port map (
        clk => clk,
        rst => rst,
        reg_read_addr1 => rs1_addr,
        reg_read_addr2 => rs2_addr,
        reg_write_addr => wb_addr,
        reg_write_data => wb_data,
        reg_write_en   => wb_en,
        reg_read_data1 => rs1_data,
        reg_read_data2 => rs2_data
    );

-- Pass PC
PC_register :   entity work.reggi
    generic map(
        N => 10
    )
    port map(
        data_in => pc_inc,
        rst       => rst,
```

```
            clk   => clk,
            data_out=> pc_inc_out
        );

end architecture structural;


-- Control Logic for RegDst
   --process(opcode)
   --begin
       --case opcode is
           --when OP_NOP =>
               --reg_dest_sel <= "00";
           --when OP_JAL | OP_JALR =>
                --reg_dest_sel <= "10"; -- Link R31
           -- R-Types
           --when OP_ADD   |
                --OP_ADDU  |
                --OP_SUB   |
                --OP_SUBU  |
                --OP_AND   |
                --OP_OR    |
                --OP_XOR   |
                --OP_SLL   |
                --OP_SRL   |
                --OP_SRA   |
                --OP_SLT   |
                --OP_SLTU  |
                --OP_SGT   |
                --OP_SGTU  |
                --OP_SLE   |
                --OP_SLEU  |
                --OP_SGE   |
                --OP_SGEU  |
                --OP_SEQ   |
                --OP_SNE   =>
               --reg_dest_sel <= "01"; -- Rd (15-11)

           -- I-Types (default)
--            when others =>
```

```
        --      reg_dest_sel <= "00"; -- Rt (20-16)
      --end case;
   --end process;
```

decode_reg.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity decode_reg is
   generic (
       DATA_WIDTH : integer := 32;
       ADDR_WIDTH : integer := 5
   );
   port (
       clk            : in  std_logic;
       rst            : in  std_logic;
       reg_read_addr1 : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
       reg_read_addr2 : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
       reg_write_addr : in  std_logic_vector(ADDR_WIDTH-1 downto 0);
       reg_write_data : in  std_logic_vector(DATA_WIDTH-1 downto 0);
       reg_write_en   : in  std_logic;
       reg_read_data1 : out std_logic_vector(DATA_WIDTH-1 downto 0);
       reg_read_data2 : out std_logic_vector(DATA_WIDTH-1 downto 0)
   );
end entity decode_reg;

architecture behavior of decode_reg is
   type reg_array_type is array (0 to 2**ADDR_WIDTH-1) of
std_logic_vector(DATA_WIDTH-1 downto 0);
   signal registers : reg_array_type := (others => (others => '0'));
begin

   -- Read Process (Asynchronous)
   reg_read_data1 <= (others => '0') when unsigned(reg_read_addr1) = 0
else
                     registers(to_integer(unsigned(reg_read_addr1)));
```

```
   reg_read_data2 <= (others => '0') when unsigned(reg_read_addr2) = 0
else
                     registers(to_integer(unsigned(reg_read_addr2)));

   -- Write Process (Synchronous)
   process(clk, rst)
   begin
       if rst = '1' then
            registers <= (others => (others => '0'));
       elsif rising_edge(clk) then
           if reg_write_en = '1' and unsigned(reg_write_addr) /= 0 then
               registers(to_integer(unsigned(reg_write_addr))) <=
reg_write_data;
           end if;
       end if;
   end process;

end architecture behavior;
```

sign_extend.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity sign_extend is
   port (
       input_data  : in  std_logic_vector(15 downto 0);
       output_data : out std_logic_vector(31 downto 0)
   );
end entity sign_extend;

architecture behavior of sign_extend is
begin
   output_data <= std_logic_vector(resize(signed(input_data), 32));
end architecture behavior;
```

tb_fetch_decode.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_fetch_decode is
end tb_fetch_decode;

architecture behavioral of tb_fetch_decode is

   component DLX_Processor
   generic (
       WIDTH       : integer := 10;
       INSTR_WIDTH : integer := 32
   );
   port (
       clk             : in  std_logic;
       rst             : in  std_logic;

       -- Branch/Jump Control
       jump_addr_in    : in  std_logic_vector(WIDTH-1 downto 0);
       pc_mux_sel      : in  std_logic;

       -- Writeback Interface
       write_addr_in   : in  std_logic_vector(4 downto 0);
       write_data_in   : in  std_logic_vector(31 downto 0);
       write_en_in     : in  std_logic;

         pc_inc                : out   std_logic_vector(WIDTH-1 downto 0);
         rs1_data          : out   std_logic_vector(INSTR_WIDTH-1 downto
0);
         rs2_data          : out   std_logic_vector(INSTR_WIDTH-1 downto
0);
         imm32                 : out std_logic_vector(INSTR_WIDTH-1 downto
0);
         instruction       : out std_logic_vector(INSTR_WIDTH-1 downto 0)
   );
   end component DLX_Processor;
```

```
    -- Clock
    signal clk : std_logic;
    constant CLK_PERIOD : time := 10 ns;

    -- Testbench signals
    signal tb_jump_addr : std_logic_vector(9 downto 0) := (others => '0');
    signal tb_pc_select : std_logic := '0';
    signal tb_rst : std_logic := '1';

    -- Writeback signals
    signal tb_wb_addr   : std_logic_vector(4 downto 0) := (others => '0');
    signal tb_wb_data   : std_logic_vector(31 downto 0) := (others => '0');
    signal tb_wb_en     : std_logic := '0';

    -- Outputs
    signal tb_pc_inc      : std_logic_vector(9 downto 0);
    signal tb_rs1_data    : std_logic_vector(31 downto 0);
    signal tb_rs2_data    : std_logic_vector(31 downto 0);
    signal tb_imm32       : std_logic_vector(31 downto 0);
    signal tb_instruction : std_logic_vector(31 downto 0);

begin

    -- Instantiate the DLX Processor
    uut : DLX_Processor
        generic map (
            WIDTH => 10,
            INSTR_WIDTH => 32
        )
        port map(
            clk             => clk,
            rst             => tb_rst,
            jump_addr_in    => tb_jump_addr,
            pc_mux_sel      => tb_pc_select,
            write_addr_in   => tb_wb_addr,
            write_data_in   => tb_wb_data,
            write_en_in     => tb_wb_en,
            pc_inc          => tb_pc_inc,
            rs1_data        => tb_rs1_data,
            rs2_data        => tb_rs2_data,
```

```
        imm32           => tb_imm32,
        instruction     => tb_instruction
    );

-- Clock process
clk_process : process
begin
    clk <= '0';
    wait for CLK_PERIOD / 2;
    clk <= '1';
    wait for CLK_PERIOD / 2;
end process;

-- Stimulus process for factorial program
stm_process : process
begin
    -- 0. Initialize Signals
    tb_wb_en <= '0';
    tb_wb_addr <= (others => '0');
    tb_wb_data <= (others => '0');

    -- 1. Reset the system to start at address 0
    tb_rst <= '1';
    wait for CLK_PERIOD * 2;
    tb_rst <= '0';
    wait for CLK_PERIOD;

    -- 2. Fetch instructions 0x000 through 0x004.
    -- 000: LW R5, n(R0)
    tb_pc_select <= '0';
    wait for CLK_PERIOD;

    -- SIMULATE WB: Instr 0 (LW R5) -> Load n=3 into R5
    tb_wb_en <= '1'; tb_wb_addr <= "00101"; tb_wb_data <= x"00000003";
    wait for CLK_PERIOD; -- 001: ADDI R4, R0, 1

    -- SIMULATE WB: Instr 1 (ADDI R4) -> R4 = 1
    tb_wb_addr <= "00100"; tb_wb_data <= x"00000001";
    wait for CLK_PERIOD; -- 002: SW R4, f(R0)
    tb_wb_en <= '0'; -- Turn off WB for SW (no WB)
```

```
        wait for CLK_PERIOD; -- 003: ADDI R7, R5, -1

        -- SIMULATE WB: Instr 3 (ADDI R7) -> R7 = 3 - 1 = 2
        tb_wb_en <= '1'; tb_wb_addr <= "00111"; tb_wb_data <= x"00000002";
        wait for CLK_PERIOD; -- 004: BEQZ R7, done
        tb_wb_en <= '0';

        wait for CLK_PERIOD;

        -- 3. At PC=0x005, instruction is JAL factorial (0x008). Jump
there.
        -- The JAL instruction works by jumping AND writing PC+1 to R31.
        -- We must simulate the Writeback of R31 here so JR R31 works
later.
        -- PC is 5, so Return Addr is 6.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000001000"; -- Jump to address 0x008

        -- SIMULATE WB: Instr 5 (JAL) -> R31 = 6
        tb_wb_en <= '1'; tb_wb_addr <= "11111"; tb_wb_data <= x"00000006";
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        tb_wb_en <= '0';
        wait for CLK_PERIOD;

        -- 4. Fetch 0x008, 0x009, 0x00A, 0x00B.
        -- 008: LW R6, f(R0) --> Load recursive result. Say 2.
        wait for CLK_PERIOD;

        -- SIMULATE WB: Instr 8 (LW R6) -> R6 = 2
        tb_wb_en <= '1'; tb_wb_addr <= "00110"; tb_wb_data <= x"00000002";
        wait for CLK_PERIOD; -- 009: ADD R8, R0, R5 (R5 is 3).

        -- SIMULATE WB: Instr 9 (ADD R8) -> R8 = 3
        tb_wb_addr <= "01000"; tb_wb_data <= x"00000003";
        wait for CLK_PERIOD; -- 00A: ADDI R3, R0, 0

        -- SIMULATE WB: Instr 10 (ADDI R3) -> R3 = 0
        tb_wb_addr <= "00011"; tb_wb_data <= x"00000000";
```

```
        wait for CLK_PERIOD; -- 00B: ADD R3, R3, R6 (R3=0, R6=2)
        tb_wb_en <= '0';

        -- 5. At PC=0x00C, instruction is008 : 04C00000; --    LW   R6,
f(R0) SUBI R8, R8, 1.
        -- SIMULATE WB: Instr 11 (ADD R3) -> R3 = 2
        tb_wb_en <= '1';
        tb_wb_addr <= "00011"; -- Write to R3
        tb_wb_data <= x"00000002";

        wait for CLK_PERIOD;
        tb_wb_en <= '0';

        -- 6. At PC=0x00D, instruction is BNEZ R8.
        -- R8 is currently 3 (set at step 4). It is NOT zero, so branch
taken.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000001011"; -- Jump to address 0x00B
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 7. We are back at 0x00B. Fetch 0x00B, 0x00C.
        -- SIMULATE WB: Instr 12 (SUBI R8) -> R8 = 3 - 1 = 2
        tb_wb_en <= '1'; tb_wb_addr <= "01000"; tb_wb_data <= x"00000002";

        wait for CLK_PERIOD * 2;
        tb_wb_en <= '0';

        -- 8. At PC=0x00D, BNEZ R8. R8 is 2. Branch Taken again usually,
but for test we say NOT taken.
        wait for CLK_PERIOD;

        -- 9. At PC=0x00E, instruction is SW. Fetch it.
        wait for CLK_PERIOD;

        -- 10. At PC=0x00F, instruction is JR R31.
        -- R31 should have 6 from our JAL shim earlier.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000000110"; -- Jump to address 0x006
```

```
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 11. At PC=0x006, instruction is SUBI R5, R5, 1.
        -- R5 was 3.
        wait for CLK_PERIOD;

        -- 12. At PC=0x007, instruction is J loop (0x004). Jump there.
        tb_pc_select <= '1';
        tb_jump_addr <= "0000000100"; -- Jump to address 0x004
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 13. At PC=0x004, BEQZ R7. R7 is 2. Not taken usually.
        -- But we force jump to done (0x010).
        tb_pc_select <= '1';
        tb_jump_addr <= "0000010000"; -- Jump to address 0x010
        wait for CLK_PERIOD;
        tb_pc_select <= '0';
        wait for CLK_PERIOD;

        -- 14. At PC=0x010, instruction is J done (itself). Fetch it.
        wait for CLK_PERIOD * 2;

        wait;
    end process;

end architecture behavioral;
```